

1) Descrivi come funziona un layout basato su flexbox e quando è preferibile usarlo rispetto al modello tradizionale. Il layout basato su flexbox si attiva impostando la proprietà `display: flex` (o `display: inline flex`) su un contenitore. Questo modello ignora il `display-outside` degli elementi contenuti e li dispone come una sequenza di riquadri (box) in riga o in colonna, a seconda della proprietà `flex-direction`. Le dimensioni dei riquadri si adattano al contenuto e allo spazio disponibile tramite proprietà come `flex-grow` e `flex-shrink`, e l'allineamento è gestito da `justify-content` e `align-items`. È preferibile usarlo quando si necessita di una disposizione flessibile che adatti le dimensioni e l'allineamento dei riquadri in modo dinamico, a differenza del modello tradizionale (`display: flow`) che dispone gli elementi come in un documento di testo.

Fonte: tweb-02-2425-ppt.pdf, slide 53-59

2) In cosa consiste l'interattività di una pagina HTML? Spiega la differenza tra interattività semplice e complessa. Una pagina HTML è interattiva quando il browser esegue del codice (quindi "fa qualcosa" che non sia la semplice navigazione) in risposta ad azioni dell'utente, modificando l'aspetto della pagina.

- **Interattività semplice:** l'effetto dell'azione dell'utente riguarda solo la visualizzazione ed è volto a migliorare la fruizione della pagina.
- **Interattività complessa:** l'effetto dell'azione comporta una vera elaborazione dati e può prevedere richieste verso un Web Server; in questo caso si parla di applicazione client-side. *Fonte: TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 164*

3) Spiega la differenza tra modello (a) Server HTTP che contiene app e modello (b) app che contiene un Server HTTP.

- **Modello (a):** Il Server HTTP (es. Apache Tomcat) è un applicativo installato che esiste di per sé e contiene un motore di esecuzione (Web Container) per tenere "accese" le app. Il Server ha il controllo, gestisce il routing e "lancia" l'applicazione smistando le richieste.
- **Modello (b):** L'applicazione ha il controllo e "lancia" il Server HTTP. Il punto di ingresso è il motore di esecuzione del linguaggio (es. Node.js), all'interno del quale l'app istanzia un Server HTTP di base e lo mette in ascolto su una porta. *Fonte: TWEB-05a-ServerSide-STAMPA.pdf, slide 39-41*

4) Descrivi il concetto di prototype chain e spiega cosa succede se una proprietà non è trovata in un oggetto. In JavaScript, gli oggetti possono ereditare proprietà da un altro oggetto che funge da prototipo. Se si cerca una proprietà in un oggetto e non la si trova, il motore di esecuzione la cerca nel suo prototipo (`__proto__`). Se non è nemmeno lì, la cerca nel prototipo del prototipo, risalendo la catena fino a `Object.prototype` (o fino a che la catena finisce). *Fonte: TWeb-02c-JavascriptEreditarieta-2526-STAMPA.pdf, slide 257, 260*

5) Spiega la differenza tra Jakarta, Spring e Spring Boot e come si relazionano tra loro.

- **Jakarta Web Profile:** È l'ambiente Java-based standard per lo sviluppo server-side che offre strumenti come le Servlet; presuppone che l'app sia contenuta in un server come Tomcat.
- **Spring:** È un framework che si basa sulle Servlet di Jakarta (usandole internamente) ma evita al programmatore la scrittura di codice ripetitivo (boilerplate), permettendo di realizzare *request handler* all'interno di classi *Controller*.
- **Spring Boot:** Incapsula Spring e un'installazione di Tomcat, fornendo pacchetti di dipendenze standard e configurazioni predefinite. Permette di creare applicazioni *stand-alone* che "contengono" il server HTTP. *Fonte:*
TWEB-05a-ServerSide-STAMPA.pdf, slide 49-55

6) Descrivi la differenza tra libreria e framework front-end e spiega in che senso React è una libreria. Una libreria è un insieme di package che si possono importare e utilizzare nel progetto. Un framework è una struttura più completa che supporta lo sviluppo a tutto tondo. React è definito una libreria perché si occupa specificamente della creazione di interfacce utente (componenti) e può essere usato da solo o integrato in framework più ampi (come Next.js), a differenza di competitor come Angular che sono framework completi. *Fonte:* *TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 338-339*

7) Descrivi come funziona il bubbling degli eventi e perché è utile in una pagina web. Il bubbling è la fase in cui un evento, dopo essere stato generato su un elemento specifico, viene propagato risalendo l'albero del DOM fino alla radice (es. da **IMG** a **DIV**, **BODY**, **HTML**). È utile perché permette di intercettare un evento (registrando un listener) su un elemento genitore anche se l'evento si è originato su uno dei suoi discendenti. *Fonte:*
TWeb-02a-JavacriptDOM-2526-STAMPA.pdf, slide 254

8) Perché React richiede che i componenti siano funzioni pure? Quali problemi evita? no (*Nota: Nelle slide fornite, pur parlando di componenti come funzioni, non è presente una spiegazione esplicita sul requisito di "funzioni pure" e sui problemi specifici che questo evita*).

9) Spiega cos'è una callback in JavaScript/TypeScript e porta un esempio pratico. Una callback è una funzione che viene passata come argomento a un'altra funzione (spesso asincrona) affinché venga richiamata ("called back") quando un'operazione è completata o si verifica un evento. *Esempio:* In Node.js, **http.createServer** prende come argomento una funzione callback che viene eseguita ogni volta che arriva una richiesta HTTP. *Fonte:*
TWEB-05c-ServerSide-STAMPA.pdf, slide 114;
TWeb-02e-FetchPromise-2526-STAMPA.pdf, slide 251

10) Confronta la tecnica di show/hide con quella di mount/unmount dei componenti figli. In quali casi useresti ciascuna?

- **Show/hide:** Il componente figlio rimane nel DOM ma viene nascosto (es. via CSS). Si usa quando il componente deve essere acceso/spento di frequente e rapidamente.
- **Mount/unmount:** Il componente viene effettivamente aggiunto o rimosso dal DOM. Si usa quando il componente è "sostanzioso" e va mostrato solo occasionalmente, o

quando la struttura dipende dai dati. *Fonte:*
TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 342

11) Illustra il concetto di gerarchia di componenti in React, distinguendo genitori e figli. Un componente UI può essere costruito a partire da altri sotto-componenti. Il componente che contiene è il "genitore", quelli contenuti sono i "figli". Il genitore gestisce i figli, decidendo ad esempio quando montarli o smontarli nel DOM. *Fonte:*
TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 341-342

12) Descrivi il ruolo del caching in un Web Server e i benefici che apporta. Il caching consiste nel memorizzare risposte HTTP associate alle relative richieste per evitare di dover reperire o calcolare nuovamente la stessa risorsa. I benefici sono la velocizzazione della risposta verso il client e la diminuzione del carico di lavoro del server stesso. *Fonte:*
TWEB-05a-ServerSide-STAMPA.pdf, slide 24

13) Quali sono i principali componenti di una HTTP Response? Fai un esempio di codice di stato e spiegate il significato. Una HTTP Response è composta da:

1. **Response line (o Start line):** contiene la versione HTTP, lo *status code* e lo *status text*.
2. **Header:** metadati come *Content-Type*, *Date*, *Server*.
3. **Body (payload):** il contenuto vero e proprio (es. HTML, JSON, immagini). *Esempio codice di stato: 404* (con status text "Not Found"), che indica che la risorsa richiesta non è stata trovata sul server (errore del client). *Fonte:*
TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 173, 178

14) Illustra il funzionamento della cascata e della specificità nei CSS con un esempio. La cascata risolve i conflitti quando più regole si applicano allo stesso elemento. Conta l'ordine: se la specificità è la stessa, vince la regola definita per ultima. L'ordine di priorità tipico è: dichiarazioni in linea > fogli di stile interni > fogli di stile esterni (dipende dall'ordine di inclusione). *Esempio:* Se un foglio esterno definisce `h1 { color: red; }` ma nell'HTML c'è `<h1 style="color: blue;">`, il testo sarà blu perché lo stile in linea ha priorità ("vince" nella cascata). *Fonte: tweb-02-2425-ppt.pdf, slide 409*

15) Confronta le unità di misura relative ed assolute in CSS e spiega in quali casi usare ciascuna.

- **Absolute (es. px, mm, in, pt):** Definiscono una dimensione fissa. Utili per dettagli grafici precisi come bordi o immagini con dimensioni note.
- **Relative (es. rem, em, %, vw, vh):** Definiscono dimensioni in proporzione ad altri elementi (es. dimensione font, larghezza viewport). Utili per layout responsivi e testi che si adattano ai dispositivi. *Fonte: tweb-02-2425-ppt.pdf, slide 412*

16) Illustra la differenza tra autenticazione e autorizzazione, portando esempi in ambito Web Server.

- **Autenticazione:** Verifica l'identità dell'utente (es. tramite login con username/password). In una "Basic Auth", si verifica se la sessione contiene l'attributo utente dopo un login corretto.

- **Autorizzazione:** Permette o vieta l'accesso a risorse basandosi sull'identità o sull'origine. *Esempio:* Un Web Server può permettere l'accesso a certe risorse solo a IP interni alla rete aziendale (autorizzazione basata su IP) o rispondere con 401 Unauthorized se l'utente non ha effettuato il login (mancata autenticazione). *Fonte:* *TWEB-05a-ServerSide-STAMPA.pdf, slide 23; TWEB-05c-ServerSide-STAMPA.pdf, slide 108*

17) Spiega la differenza tra una richiesta HTTP GET e POST, fornendo un esempio pratico.

- **GET:** Usata per ottenere informazioni/risorse dal server. Non ha un body. *Esempio:* Il browser richiede la pagina di un libro (**GET /bookshelf/books/12**).
- **POST:** Usata per inviare dati al server per l'elaborazione o la memorizzazione. Ha un body contenente i dati. *Esempio:* Inviare i dati di un nuovo libro da inserire (**POST /bookshelf/books** con dati JSON nel body). *Fonte:* *TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 140, 175; TWEB-05a-ServerSide-STAMPA.pdf, slide 33*

18) Descrivi la struttura di una URL e spiega il ruolo della query string e del fragment.

Struttura: **protocol://hostname[:port]/path[?querystring][#fragment]**.

- **Query string:** Inizia con **?** e contiene coppie chiave/valore separate da **&**. Serve ad aggiungere parametri opzionali alla richiesta (es. filtri di ricerca: **?min=5&sort=alpha**).
- **Fragment:** Inizia con **#**. Indica una specifica posizione (location) all'interno della risorsa (pagina HTML) ottenuta, a cui il browser deve posizionarsi. *Fonte:* *TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 142-145*

19) Descrivi il ciclo del browser dal caricamento di una pagina HTML fino alla gestione degli eventi.

1. Il browser ottiene la pagina HTML tramite una transazione HTTP.
2. Richiede le risorse aggiuntive (CSS, media, JS).
3. Renderizza la pagina.
4. Esegue gli script JavaScript (inizializza l'ambiente).
5. Attende ed esegue funzioni (listener) in risposta a eventi (utente o di rete).
6. Se l'esecuzione modifica il DOM, il browser ri-renderizza le parti modificate. *Fonte:* *TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 168-172*

20) Descrivi il ruolo del middleware in Express e come può influire sulla sicurezza di un'applicazione. Il middleware è costituito da funzioni eseguite prima che la richiesta arrivi all'handler principale. Può trasformare la richiesta, eseguire elaborazioni preliminari o bloccarla. Influisce sulla sicurezza fungendo da filtro: può respingere richieste non autorizzate, validare input o gestire l'autenticazione prima che il "core" dell'app venga coinvolto. *Fonte:* *TWEB-05a-ServerSide-STAMPA.pdf, slide 44; TWEB-05c-ServerSide-STAMPA.pdf, slide 120*

21) Spiega la differenza tra elementi a blocco e in linea, fornendo esempi pratici di ciascuno.

- **Elementi a blocco:** Definiscono un riquadro che occupa tutto lo spazio orizzontale disponibile e vanno a capo. Esempi: `DIV`, `P`, `H1`, `SECTION`.
- **Elementi in linea:** Seguono il flusso del testo e occupano solo lo spazio necessario al contenuto. Esempi: `SPAN`, `IMG`, `A`, `EM`. Fonte: *tweb-02-2425-ppt.pdf, slide 382-383, 433*

22) Confronta i tre modi di dichiarare funzioni in JavaScript (named, anonymous, arrow) e spiega vantaggi e svantaggi.

1. **Named:** `function nome(p) { ... }`. Vantaggio: chiara identificazione.
2. **Anonymous:** `const nome = function(p) { ... }`. Assegnata a variabile.
3. **Arrow:** `const nome = (p) => { ... }`. Vantaggio: sintassi concisa.
Svantaggio/Differenza: non possiede un proprio `this` (non è attivo il binding di `this` all'oggetto proprietario se usata come metodo, eredita il `this` dal contesto lessicale).
Fonte: *TWeb-02a-JavacriptBase-2526-STAMPA.pdf, slide 211-213*

23) Che differenza c'è tra undefined e null in JavaScript? Porta un esempio pratico di utilizzo. Sono entrambi tipi base.

- **undefined:** Indica che una variabile non è stata inizializzata o che un parametro di funzione non è stato passato. Esempio: In `function myfun(x, y)`, se chiamo `myfun(1)`, `y` sarà `undefined`.
- **null:** Indica l'assenza intenzionale di un valore o un oggetto non trovato. Esempio: `document.getElementById("nonEsiste")` restituisce `null`. Fonte: *TWeb-02a-JavacriptBase-2526-STAMPA.pdf, slide 201, 211; TWeb-02a-JavacriptDOM-2526-STAMPA.pdf, slide 239*

24) Scrivi un esempio di componente React con props e mostra come il genitore passa valori differenti a istanze diverse. Componente Figlio:

```
interface CompProps { nome: string; }  
function Comp(props: CompProps) { return <p>Ciao {props.nome}</p>; }
```

Componente Genitore:

```
function Genitore() {  
  return (  
    <div>  
      <Comp nome="Merlino"/>  
      <Comp nome="Gandalf"/>  
    </div>  
  );  
}
```

Fonte: TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 356

25) Cosa fa un Rewrite Engine e perché è importante in combinazione con routing client-side? Un Rewrite Engine traduce le URL richieste dal client in pathname interni del server. È importante per il routing client-side (SPA) perché se l'utente digita manualmente una URL specifica dell'app (es. `/app/profilo`), il browser la chiede al server. Senza rewrite, il server darebbe 404 (la risorsa non esiste fisicamente); il Rewrite Engine invece deve redirigere la richiesta alla pagina principale dell'applicazione (es. `index.html`) affinché il router client-side possa poi gestire la vista corretta. Fonte: TWEB-05a-ServerSide-STAMPA.pdf, slide 27-28

26) Parlami del ciclo di vita di una servlet e di come il container lo gestisce. Il Web Container gestisce la servlet tramite tre metodi:

1. **Init:** Invocato alla creazione (avvio server o primo utilizzo). Chiama `init(config)`.
2. **Service:** Invocato per ogni richiesta. `service(req, res)` smista la richiesta ai metodi `doGet`, `doPost`, ecc.
3. **Destroy:** Invocato allo spegnimento o rimozione dell'app per fare pulizia. Fonte: TWEB-05c-ServerSide-STAMPA.pdf, slide 127-133

27) Spiega cosa si intende per front-end e back-end in un'applicazione web, e perché spesso si parla di architettura a strati.

- **Front-end:** Strato lato client che gestisce l'interazione con l'utente (User Interface).
- **Back-end:** Strato (tipicamente lato server) che gestisce la persistenza dei dati e la logica applicativa profonda. Si parla di architettura a strati perché ogni livello offre servizi a quello superiore nascondendo i dettagli implementativi, permettendo modularità e gestione separata dei componenti. Fonte: TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 190-192

28) Spiega le differenze tra modello single-thread event loop e modello multi-thread nella gestione delle richieste HTTP.

- **Multi-thread (es. Tomcat):** Il server genera un thread separato per ogni richiesta. Può gestire le richieste in modo sincrono all'interno del thread.
- **Single-thread event loop (es. Node.js):** Il server usa un unico thread per gestire la coda delle richieste sequenzialmente. Le operazioni lunghe (I/O, calcoli) devono essere asincrone per non bloccare il loop. Fonte: TWEB-05a-ServerSide-STAMPA.pdf, slide 13-16

29) Cosa si intende per risorsa statica e risorsa dinamica? Porta un esempio di ciascuna.

- **Risorsa statica:** Un file già esistente restituito così com'è dal server. Esempio: un file immagine `.jpg` o un file HTML pre-scritto.
- **Risorsa dinamica:** Una risorsa calcolata al momento dal server eseguendo un programma. Esempio: il risultato di una ricerca o una pagina HTML generata in base ai dati di un database. Fonte: TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 155-156

30) Spiega il pattern MVVM e come React implementa property binding ed event binding. MVVM (Model-View-ViewModel) separa la View (DOM) dal Model (Dati) tramite il ViewModel (Componente) che gestisce la logica.

- **Property binding:** Il passaggio di dati dal ViewModel alla View. In React si fa passando variabili (es. state) come attributi o contenuto interpolato (`value={stateVar}`).
- **Event binding:** Il collegamento di eventi della View al codice. In React si assegnano funzioni callback agli eventi (`onChange={handleChange}`). React realizza un binding bidirezionale combinando questi due meccanismi manualmente. *Fonte: TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 346-347, 363*

31) Descrivi il ruolo delle annotazioni @Entity e @Repository in Spring JPA.

- **@Entity:** Annota una classe per indicare che corrisponde a una tabella del DB e che le sue istanze sono gestite da JPA.
- **@Repository:** Annota un'interfaccia che estende `JpaRepository`, fornendo automaticamente metodi per le operazioni CRUD (save, find, delete) sull'entità specificata. *Fonte: TWEB-05b-ServerSide-STAMPA.pdf, slide 86-88*

32) Confronta i tre tipi di fogli di stile (esterno, interno, inline) e discuti i pro e contro di ciascuno.

- **Esterno (link):** File separato `.css`. Pro: riutilizzabile su più pagine, pulizia del codice.
- **Interno (tag style):** Nel blocco `<head>`. Pro: tutto in un file. Contro: non riutilizzabile altrove.
- **Inline (attributo style):** Sugli elementi. Pro: vince nella cascata per modifiche puntuali. Contro: mescola contenuto e presentazione, difficile da mantenere. *Fonte: tweb-02-2425-ppt.pdf, slide 409*

33) Illustra cosa fa JDBC e perché rende indipendente l'accesso ai dati dal tipo di database. JDBC (Java Database Connectivity) è un insieme di classi e interfacce standard. Fornisce un'API unica per l'accesso ai dati. L'indipendenza è garantita dai **driver** specifici per ogni DBMS (Postgres, MySQL) che implementano queste interfacce: il codice Java usa l'interfaccia standard JDBC e basta cambiare il driver per cambiare DB. *Fonte: TWEB-05b-ServerSide-STAMPA.pdf, slide 82-83*

34) Descrivi il ruolo del DOM nel ciclo di vita di una pagina e spiega perché viene definito una struttura ad albero. Il DOM (Document Object Model) è la struttura dati in memoria che rappresenta la pagina web, inizializzata da HTML e CSS. È definito ad albero perché rispecchia la struttura "a scatole cinesi" dell'HTML: c'è una radice (`html`), nodi intermedi (elementi contenitori come `body`, `div`) e foglie (testo, immagini). È l'interfaccia tramite cui gli script modificano la pagina. *Fonte: TWeb-02a-JavacriptDOM-2526-STAMPA.pdf, slide 228-229*

35) Illustra la differenza tra export default e export standard nei moduli JavaScript.

- **Standard export:** Si usa `export` davanti a dichiarazioni (es. `export const x`). Permette di esportare più elementi. Si importano con le graffe: `import {x} from ....`
- **Default export:** Si usa `export default` per esportare un singolo elemento principale del modulo. Si importa senza graffe e con un nome a scelta: `import AnyName from ....` Fonte: *TWeb-02a-JavascriptBase-2526-STAMPA.pdf, slide 225-226*

36) Cos'è una closure e in quali casi pratici può essere utile? Una closure si verifica quando una funzione definita all'interno di un'altra continua ad avere accesso alle variabili dello scope della funzione esterna anche dopo che quest'ultima è terminata. È utile per creare funzioni che mantengono uno stato privato o per configurare comportamenti specifici (es. function creators). Fonte: *TWeb-02a-JavascriptBase-2526-STAMPA.pdf, slide 216*

37) Descrivi i passi fondamentali per modificare dinamicamente il DOM con JavaScript.

1. **Trovare** l'elemento (navigazione) usando metodi come `getElementById` o `querySelector`.
2. **Modificare** l'elemento (contenuti/attributi) usando proprietà come `innerHTML`, `classList` o metodi come `setAttribute`.
3. **Aggiungere/Rimuovere** elementi usando `createElement`, `insertAdjacentElement` o `remove`. Fonte: *TWeb-02a-JavascriptDOM-2526-STAMPA.pdf, slide 235, 244-247*

38) Spiega la differenza tra linguaggi compilati e interpretati, portando esempi concreti.

- **Compilati (es. C):** Il codice sorgente viene tradotto interamente in linguaggio macchina (eseguibile) da un compilatore prima dell'esecuzione.
- **Interpretati (es. JavaScript):** Il codice viene eseguito direttamente da un motore (interprete/macchina virtuale) istruzione per istruzione, senza produrre un file eseguibile a sé stante. Fonte: *TWeb-02a-JavascriptBase-2526-STAMPA.pdf, slide 196*

39) Descrivere il ciclo di vita delle @Entity nella Java Persistence API di Spring

Un'entità inizia come **New** (creata in memoria, non gestita). Quando viene salvata (`repo.save(e)`), diventa **Managed** (gestita da JPA, sincronizzata col DB). Resta managed fino alla fine della transazione. Può diventare **Detached** se il contesto di persistenza viene chiuso. Fonte: *TWEB-05b-ServerSide-STAMPA.pdf, slide 91-92*

40) Delineare le principali funzionalità di un Web Server. Fornire contenuti statici e dinamici tramite HTTP, Logging, Gestione di host virtuali, Autorizzazione e autenticazione, Caching, Riduzione di banda (throttling), Riscrittura delle URL (Rewrite Engine). Fonte: *TWEB-05a-ServerSide-STAMPA.pdf, slide 19*

41) Illustrare il pattern architetturale MVVM per come è stato presentato nel contesto di React. In React, il pattern MVVM vede il DOM come la **View**, i dati dell'applicazione come

il **Model**, e i Componenti React come il **ViewModel**. Il componente definisce l'aspetto (template) e il comportamento, gestendo la comunicazione tra dati e vista tramite property binding (passaggio di dati/state alla view) ed event binding (callback che reagiscono agli eventi aggiornando i dati). *Fonte: TWeb-ReactTeoria-04-2526-ppt-STAMPA.pdf, slide 346-348*

42) Illustrare le macrocategorie in cui è possibile suddividere le funzionalità offerte dalla DOM API del browser. Le funzionalità si dividono in tre macrocategorie:

1. **Navigazione/Attraversamento:** Trovare elementi (`getElementById`, `querySelector`, `children`).
2. **Modifica:** Cambiare struttura, attributi o stili (`createElement`, `innerHTML`, `classList`).
3. **Gestione Eventi:** Rispondere alle interazioni (`addEventListener`). *Fonte: TWeb-02a-JavascriptDOM-2526-STAMPA.pdf, slide 235*

43) Descrivere le caratteristiche del linguaggio Typescript in quanto estensione di Javascript (parlando quindi delle caratteristiche che lo distinguono o che gli sono proprie, non di quelle che “eredita” da Javascript stesso). TypeScript aggiunge a JavaScript un **sistema di tipi statico** (dichiarazioni di tipo per variabili, parametri, ritorni), **inferenza dei tipi** e costrutti come **Interfacce** e **Generics**. È un linguaggio che si **compila** (transpila) in JavaScript; i tipi sono usati solo in fase di sviluppo/compilazione e scompaiono a runtime. *Fonte: TWeb-02d-Typescript-2526-STAMPA.pdf, slide 261, 276*

44) Descrivere le fasi che compongono un ciclo del browser, a partire da quando esso ottiene HTML in risposta a una precedente transazione HTTP.

1. Il browser analizza l'HTML e determina se servono altre risorse (CSS, JS, media).
2. Richiede queste risorse.
3. Renderizza la pagina (visualizzazione) usando HTML e CSS.
4. Esegue gli script JavaScript (inizializza l'ambiente).
5. Entra in attesa di eventi (es. click utente, risposte di rete).
6. Al verificarsi di un evento, esegue la funzione associata e, se necessario, ri-renderizza la pagina. *Fonte: TWeb-01-IntroTeoria-2526-STAMPA.pdf, slide 168-172*